Agno Framework - Complete API Documentation

This comprehensive documentation covers all public APIs, functions, and components of the Agno framework.

Table of Contents

1. Introduction to Agno

Agno is a multi-agent framework, runtime and control plane built for speed, privacy, and scale. It provides tools for building:

- **Agents** with memory, knowledge, session management, and advanced features
- **Multi-Agent Teams** that operate autonomously under a team leader
- **Step-based Workflows** for controlled, deterministic execution

- **AgentOS** - A ready-to-use FastAPI app for serving agents in production

Key Features

- **Model Agnostic**: Works with any model provider
- **Type Safe**: Enforce structured I/O through input_schema and output_schema
- **Persistent Storage**: Give your Agents, Teams, and Workflows a database
- **Agentic RAG**: Connect to 20+ vector stores with hybrid search + reranking
- **Human-in-the-Loop**: Native support for confirmations and manual overrides
- **Guardrails**: Built-in safeguards for validation and security
- **MCP Integration**: First-class support for Model Context Protocol
- **100+ Toolkits**: Built-in toolkits with thousands of tools

2. Core Concepts

2.1 Agents

An Agent is the fundamental building block in Agno. It represents an AI assistant that can: - Process user queries - Use tools to perform actions - Access knowledge bases - Maintain conversation history - Store and recall user memories

2.2 Teams

A Team is a collection of Agents (or sub-teams) that work together to accomplish tasks. Teams operate autonomously with a team leader that maintains shared state and context.

2.3 Workflows

Workflows are step-based execution systems for controlled, deterministic execution. Steps can be Agents, Teams, or regular Python functions that run sequentially, in parallel, in loops, branches, or conditionally.

2.4 Knowledge

Knowledge is Agno's way of providing Agents with information they can search at runtime. It connects to 20+ vector stores with hybrid search and reranking capabilities.

2.5 Sessions

Sessions track conversations and maintain state across multiple interactions. Each session can have its own state, history, and metadata.

2.6 Memory

Memory allows Agents to store insights and facts about users that persist across sessions. This enables personalization and context retention.

3. Agent API Reference

3.1 Agent Class

The `Agent` class is the core component for creating AI assistants.

Import

```
from agno.agent import Agent
```

Basic Agent Creation

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions="You are a helpful assistant",
    markdown=True,
)
```

Agent Parameters

Core Parameters

- **model** (Model, required): The language model to use
- **name** (str, optional): Name of the agent
- **id** (str, optional): Unique identifier (auto-generated if not set)
- **instructions** (str | list[str], optional): Instructions for the agent
- **description** (str, optional): Description of the agent's purpose
- **markdown** (bool, default=False): Enable markdown formatting in responses

Session Parameters

- **user_id** (str, optional): Default user ID for this agent
- **session_id** (str, optional): Default session ID (auto-generated if not set)
- **session_state** (dict, optional): Default session state
- **add_session_state_to_context** (bool, default=False): Add session state to context
- **enable_agentic_state** (bool, default=False): Give agent tools to update state
- **cache_session** (bool, default=False): Cache session in memory

History Parameters

- **add_history_to_context** (bool, default=False): Add conversation history to context
- **num_history_runs** (int, optional): Number of history runs to include
- **search_session_history** (bool, default=False): Search previous sessions

Knowledge Parameters

- **knowledge** (Knowledge, optional): Knowledge base to use
- **search_knowledge** (bool, default=False): Enable knowledge search
- **num_documents** (int, optional): Number of documents to retrieve
- **rerank** (bool, default=False): Enable reranking of results

Tool Parameters

- **tools** (list[Toolkit | Function], optional): Tools available to the agent
- **tool_choice** (str, optional): Tool selection strategy

Memory Parameters

- **memory** (MemoryManager, optional): Memory manager for user memories
- **add_memories_to_context** (bool, default=False): Add memories to context

Output Parameters

- **input_schema** (BaseModel, optional): Input validation schema
- **output_schema** (BaseModel, optional): Output validation schema

Other Parameters

- **db** (BaseDb, optional): Database for persistence
- **guardrails** (list[BaseGuardrail], optional): Guardrails for validation
- **reasoning** (bool, default=False): Enable reasoning mode
- **show_tool_calls** (bool, default=True): Show tool calls in output
- **debug_mode** (bool, default=False): Enable debug logging

Agent Methods

run()

Execute the agent synchronously.

```
response = agent.run("What is the weather today?")
print(response.content)
```

Parameters: - **message** (str | Message | list[Message]): The input message - **user_id** (str, optional): User ID for this run - **session_id** (str, optional): Session ID for this run - **stream** (bool, default=False): Stream the response - ****kwargs**: Additional parameters

Returns: RunOutput object with content, messages, and metadata

arun()

Execute the agent asynchronously.

```
response = await agent.arun("What is the weather today?")
print(response.content)
```

Parameters: Same as run()

Returns: RunOutput object

print_response()

Print the agent's response with formatting.

```
agent.print_response("Hello!", stream=True)
```

Parameters: - **message** (str): The input message - **stream** (bool, default=False): Stream the response - ****kwargs**: Additional parameters

3.2 Agent Examples from Cookbook

Basic Agent

File: `01_basic_agent.py`

```
"""🗽 Basic Agent Example - Creating a Quirky News Reporter

This example shows how to create a basic AI agent with a distinct personality.
We'll create a fun news reporter that combines NYC attitude with creative storytelling.
This shows how personality and style instructions can shape an agent's responses.

Run `pip install openai agno` to install dependencies.
"""

from textwrap import dedent

from agno.agent import Agent
from agno.models.openai import OpenAIChat

# Create our News Reporter with a fun personality
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions=dedent("""\
        You are an enthusiastic news reporter with a flair for storytelling! 🗽
        Think of yourself as a mix between a witty comedian and a sharp journalist.

        Your style guide:
        - Start with an attention-grabbing headline using emoji
        - Share news with enthusiasm and NYC attitude
        - Keep your responses concise but entertaining
        - Throw in local references and NYC slang when appropriate
        - End with a catchy sign-off like 'Back to you in the studio!' or 'Reporting li

        Remember to verify all facts while keeping that NYC energy high!\
    """),
    markdown=True,
)

# Example usage
agent.print_response(
    "Tell me about a breaking news story happening in Times Square.", stream=True
)

# More example prompts to try:
"""
```

```
Try these fun scenarios:
1. "What's the latest food trend taking over Brooklyn?"
2. "Tell me about a peculiar incident on the subway today"
3. "What's the scoop on the newest rooftop garden in Manhattan?"
4. "Report on an unusual traffic jam caused by escaped zoo animals"
5. "Cover a flash mob wedding proposal at Grand Central"
"""
```

Agent with Tools

File: `02_agent_with_tools.py`

```
"""🗽 Agent with Tools - Your AI News Buddy that can search the web

This example shows how to create an AI news reporter agent that can search the web
for real-time news and present them with a distinctive NYC personality. The agent combi
web searching capabilities with engaging storytelling to deliver news in an entertainin

Run `pip install openai ddgs agno` to install dependencies.
"""

from textwrap import dedent

from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools

# Create a News Reporter Agent with a fun personality
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions=dedent("""\
        You are an enthusiastic news reporter with a flair for storytelling! 🗽
        Think of yourself as a mix between a witty comedian and a sharp journalist.

        Follow these guidelines for every report:
        1. Start with an attention-grabbing headline using relevant emoji
        2. Use the search tool to find current, accurate information
        3. Present news with authentic NYC enthusiasm and local flavor
        4. Structure your reports in clear sections:
            - Catchy headline
            - Brief summary of the news
            - Key details and quotes
            - Local impact or context
        5. Keep responses concise but informative (2-3 paragraphs max)
        6. Include NYC-style commentary and local references
        7. End with a signature sign-off phrase

        Sign-off examples:
        - 'Back to you in the studio, folks!'
        - 'Reporting live from the city that never sleeps!'
        - 'This is [Your Name], live from the heart of Manhattan!'
```

```
        Remember: Always verify facts through web searches and maintain that authentic
    """),
    tools=[DuckDuckGoTools()],
    markdown=True,
)

# Example usage
agent.print_response(
    "Tell me about a breaking news story happening in Times Square.", stream=True
)

# More example prompts to try:
"""
Try these engaging news queries:
1. "What's the latest development in NYC's tech scene?"
2. "Tell me about any upcoming events at Madison Square Garden"
3. "What's the weather impact on NYC today?"
4. "Any updates on the NYC subway system?"
5. "What's the hottest food trend in Manhattan right now?"
"""
```

Agent with Knowledge

File: `03_agent_with_knowledge.py`

```
"""🧠 Agent with Knowledge - Your AI Cooking Assistant!

This example shows how to create an AI cooking assistant that combines knowledge from a
curated recipe database with web searching capabilities. The agent uses a PDF knowledge
of authentic Thai recipes and can supplement this information with web searches when ne

Run `pip install openai lancedb tantivy pypdf ddgs agno` to install dependencies.
"""

from textwrap import dedent

from agno.agent import Agent
from agno.knowledge.embedder.openai import OpenAIEmbedder
from agno.knowledge.knowledge import Knowledge
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.vectordb.lancedb import LanceDb, SearchType

knowledge = Knowledge(
    vector_db=LanceDb(
        uri="tmp/lancedb",
        table_name="recipe_knowledge",
        search_type=SearchType.hybrid,
        embedder=OpenAIEmbedder(id="text-embedding-3-small"),
    ),
)

knowledge.add_content(
    url="https://agno-public.s3.amazonaws.com/recipes/ThaiRecipes.pdf",
)

# Create a Recipe Expert Agent with knowledge of Thai recipes
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions=dedent("""\
        You are a passionate and knowledgeable Thai cuisine expert! 👨‍🍳
        Think of yourself as a combination of a warm, encouraging cooking instructor,
        a Thai food historian, and a cultural ambassador.

        Follow these steps when answering questions:
```

```
        1. If the user asks a about Thai cuisine, ALWAYS search your knowledge base for
        2. If the information in the knowledge base is incomplete OR if the user asks a
        3. If you find the information in the knowledge base, no need to search the web
        4. Always prioritize knowledge base information over web results for authentici
        5. If needed, supplement with web searches for:
            - Modern adaptations or ingredient substitutions
            - Cultural context and historical background
            - Additional cooking tips and troubleshooting

        Communication style:
        1. Start each response with a relevant cooking emoji
        2. Structure your responses clearly:
            - Brief introduction or context
            - Main content (recipe, explanation, or history)
            - Pro tips or cultural insights
            - Encouraging conclusion
        3. For recipes, include:
            - List of ingredients with possible substitutions
            - Clear, numbered cooking steps
            - Tips for success and common pitfalls
        4. Use friendly, encouraging language

        Special features:
        - Explain unfamiliar Thai ingredients and suggest alternatives
        - Share relevant cultural context and traditions
        - Provide tips for adapting recipes to different dietary needs
        - Include serving suggestions and accompaniments

        End each response with an uplifting sign-off like:
        - 'Happy cooking!            (Enjoy your meal)!'
        - 'May your Thai cooking adventure bring joy!'
        - 'Enjoy your homemade Thai feast!'

        Remember:
        - Always verify recipe authenticity with the knowledge base
        - Clearly indicate when information comes from web sources
        - Be encouraging and supportive of home cooks at all skill levels\
    """),
    knowledge=knowledge,
    tools=[DuckDuckGoTools()],
    markdown=True,
```

```
)

agent.print_response(
    "How do I make chicken and galangal in coconut milk soup", stream=True
)
agent.print_response("What is the history of Thai curry?", stream=True)
agent.print_response("What ingredients do I need for Pad Thai?", stream=True)

# More example prompts to try:
"""
Explore Thai cuisine with these queries:
1. "What are the essential spices and herbs in Thai cooking?"
2. "Can you explain the different types of Thai curry pastes?"
3. "How do I make mango sticky rice dessert?"
4. "What's the proper way to cook Thai jasmine rice?"
5. "Tell me about regional differences in Thai cuisine"
"""
```

Structured Output

File: `05_structured_output.py`

```
"""🎬 Agent with Structured Output - Your AI Movie Script Generator

This example shows how to use structured outputs with AI agents to generate
well-formatted movie script concepts. It shows two approaches:
1. JSON Mode: Traditional JSON response parsing
2. Structured Output: Enhanced structured data handling

Run `pip install openai agno` to install dependencies.
"""

from textwrap import dedent
from typing import List

from agno.agent import Agent, RunOutput  # noqa
from agno.models.openai import OpenAIChat
from pydantic import BaseModel, Field


class MovieScript(BaseModel):
    setting: str = Field(
        ...,
        description="A richly detailed, atmospheric description of the movie's primary
    )
    ending: str = Field(
        ...,
        description="The movie's powerful conclusion that ties together all plot threac
    )
    genre: str = Field(
        ...,
        description="The film's primary and secondary genres (e.g., 'Sci-fi Thriller',
    )
    name: str = Field(
        ...,
        description="An attention-grabbing, memorable title that captures the essence c
    )
    characters: List[str] = Field(
        ...,
        description="4-6 main characters with distinctive names and brief role descript
    )
    storyline: str = Field(
```

```
        ...,
        description="A compelling three-sentence plot summary: Setup, Conflict, and Sta
    )


# Agent that uses JSON mode
json_mode_agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    description=dedent("""\
        You are an acclaimed Hollywood screenwriter known for creating unforgettable bl
        With the combined storytelling prowess of Christopher Nolan, Aaron Sorkin, and
        you craft unique stories that captivate audiences worldwide.

        Your specialty is turning locations into living, breathing characters that driv
    """),
    instructions=dedent("""\
        When crafting movie concepts, follow these principles:

        1. Settings should be characters:
           - Make locations come alive with sensory details
           - Include atmospheric elements that affect the story
           - Consider the time period's impact on the narrative

        2. Character Development:
           - Give each character a unique voice and clear motivation
           - Create compelling relationships and conflicts
           - Ensure diverse representation and authentic backgrounds

        3. Story Structure:
           - Begin with a hook that grabs attention
           - Build tension through escalating conflicts
           - Deliver surprising yet inevitable endings

        4. Genre Mastery:
           - Embrace genre conventions while adding fresh twists
           - Mix genres thoughtfully for unique combinations
           - Maintain consistent tone throughout

        Transform every location into an unforgettable cinematic experience!\
    """),
    output_schema=MovieScript,
```

```
    use_json_mode=True,
)

# Agent that uses structured outputs
structured_output_agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    description=dedent("""\
        You are an acclaimed Hollywood screenwriter known for creating unforgettable bl
        With the combined storytelling prowess of Christopher Nolan, Aaron Sorkin, and
        you craft unique stories that captivate audiences worldwide.

        Your specialty is turning locations into living, breathing characters that driv
    """),
    instructions=dedent("""\
        When crafting movie concepts, follow these principles:

        1. Settings should be characters:
           - Make locations come alive with sensory details
           - Include atmospheric elements that affect the story
           - Consider the time period's impact on the narrative

        2. Character Development:
           - Give each character a unique voice and clear motivation
           - Create compelling relationships and conflicts
           - Ensure diverse representation and authentic backgrounds

        3. Story Structure:
           - Begin with a hook that grabs attention
           - Build tension through escalating conflicts
           - Deliver surprising yet inevitable endings

        4. Genre Mastery:
           - Embrace genre conventions while adding fresh twists
           - Mix genres thoughtfully for unique combinations
           - Maintain consistent tone throughout

        Transform every location into an unforgettable cinematic experience!\
    """),
    output_schema=MovieScript,
)
```

```
# Example usage with different locations
json_mode_agent.print_response("Tokyo", stream=True)
structured_output_agent.print_response("Ancient Rome", stream=True)

# More examples to try:
"""
Creative location prompts to explore:
1. "Underwater Research Station" - For a claustrophobic sci-fi thriller
2. "Victorian London" - For a gothic mystery
3. "Dubai 2050" - For a futuristic heist movie
4. "Antarctic Research Base" - For a survival horror story
5. "Caribbean Island" - For a tropical adventure romance
"""

# To get the response in a variable:
# from rich.pretty import pprint

# json_mode_response: RunOutput = json_mode_agent.run("New York")
# pprint(json_mode_response.content)
# structured_output_response: RunOutput = structured_output_agent.run("New York")
# pprint(structured_output_response.content)
```

4. Team API Reference

4.1 Team Class

The `Team` class enables multiple agents to work together autonomously.

Import

```
from agno.team.team import Team
```

Basic Team Creation

```
from agno.agent import Agent
from agno.team.team import Team
from agno.models.openai import OpenAIChat

# Create specialized agents
web_agent = Agent(
    name="Researcher",
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
)

writer_agent = Agent(
    name="Writer",
    model=OpenAIChat(id="gpt-4o"),
)

# Create team
team = Team(
    members=[web_agent, writer_agent],
    model=OpenAIChat(id="gpt-4o"),
    instructions="Research and write articles",
)
```

Team Parameters

- **members** (list[Agent | Team], required): List of agents or sub-teams
- **model** (Model, required): Model for the team leader
- **name** (str, optional): Name of the team
- **id** (str, optional): Unique identifier
- **instructions** (str | list[str], optional): Instructions for the team
- **db** (BaseDb, optional): Database for persistence
- **show_members_responses** (bool, default=True): Show individual agent responses
- **markdown** (bool, default=False): Enable markdown formatting

Team Methods

run()

Execute the team synchronously.

```
response = team.run("Research and write about AI")
print(response.content)
```

arun()

Execute the team asynchronously.

```
response = await team.arun("Research and write about AI")
```

Team Example from Cookbook

```
"""🗞️ Multi-Agent Team - Your Professional News & Finance Squad!

This example shows how to create a powerful team of AI agents working together
to provide comprehensive financial analysis and news reporting. The team consists of:
1. Web Agent: Searches and analyzes latest news
2. Finance Agent: Analyzes financial data and market trends
3. Lead Editor: Coordinates and combines insights from both agents

Run: `pip install openai ddgs yfinance agno` to install the dependencies
"""

from textwrap import dedent

from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.team.team import Team
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.tools.exa import ExaTools

web_agent = Agent(
    name="Web Agent",
    role="Search the web for information",
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
    instructions=dedent("""\
        You are an experienced web researcher and news analyst! 🔍

        Follow these steps when searching for information:
        1. Start with the most recent and relevant sources
        2. Cross-reference information from multiple sources
        3. Prioritize reputable news outlets and official sources
        4. Always cite your sources with links
        5. Focus on market-moving news and significant developments

        Your style guide:
        - Present information in a clear, journalistic style
        - Use bullet points for key takeaways
        - Include relevant quotes when available
        - Specify the date and time for each piece of news
        - Highlight market sentiment and industry trends
```

```
        - End with a brief analysis of the overall narrative
        - Pay special attention to regulatory news, earnings reports, and strategic ann
    """),
    markdown=True,
)

finance_agent = Agent(
    name="Finance Agent",
    role="Get financial data",
    model=OpenAIChat(id="gpt-4o"),
    tools=[
        ExaTools(
            include_domains=["trendlyne.com"],
            text=False,
            show_results=True,
        )
    ],
    instructions=dedent("""\
        You are a skilled financial analyst with expertise in market data! 📊

        Follow these steps when analyzing financial data:
        1. Start with the latest stock price, trading volume, and daily range
        2. Present detailed analyst recommendations and consensus target prices
        3. Include key metrics: P/E ratio, market cap, 52-week range
        4. Analyze trading patterns and volume trends
        5. Compare performance against relevant sector indices

        Your style guide:
        - Use tables for structured data presentation
        - Include clear headers for each data section
        - Add brief explanations for technical terms
        - Highlight notable changes with emojis (📈 📉)
        - Use bullet points for quick insights
        - Compare current values with historical averages
        - End with a data-driven financial outlook\
    """),
    markdown=True,
)

agent_team = Team(
    members=[web_agent, finance_agent],
```

```
    model=OpenAIChat(id="gpt-4o"),
    instructions=dedent("""\
        You are the lead editor of a prestigious financial news desk! 📰

        Your role:
        1. Coordinate between the web researcher and financial analyst
        2. Combine their findings into a compelling narrative
        3. Ensure all information is properly sourced and verified
        4. Present a balanced view of both news and data
        5. Highlight key risks and opportunities

        Your style guide:
        - Start with an attention-grabbing headline
        - Begin with a powerful executive summary
        - Present financial data first, followed by news context
        - Use clear section breaks between different types of information
        - Include relevant charts or tables when available
        - Add 'Market Sentiment' section with current mood
        - Include a 'Key Takeaways' section at the end
        - End with 'Risk Factors' when appropriate
        - Sign off with 'Market Watch Team' and the current date\
    """),
    add_datetime_to_context=True,
    markdown=True,
    show_members_responses=False,
)

# Example usage with diverse queries
agent_team.print_response(
    input="Summarize analyst recommendations and share the latest news for NVDA",
    stream=True,
)
agent_team.print_response(
    input="What's the market outlook and financial performance of AI semiconductor comp
    stream=True,
)
agent_team.print_response(
    input="Analyze recent developments and financial performance of TSLA",
    stream=True,
)
```

```
# More example prompts to try:
"""
Advanced queries to explore:
1. "Compare the financial performance and recent news of major cloud providers (AMZN, M
2. "What's the impact of recent Fed decisions on banking stocks? Focus on JPM and BAC"
3. "Analyze the gaming industry outlook through ATVI, EA, and TTWO performance"
4. "How are social media companies performing? Compare META and SNAP"
5. "What's the latest on AI chip manufacturers and their market position?"
"""
```

5. Workflow API Reference

5.1 Workflow Class

The `Workflow` class enables step-based, deterministic execution.

Import

```
from agno.workflow.workflow import Workflow
```

Basic Workflow Creation

```
from agno.workflow.workflow import Workflow
from agno.db.sqlite import SqliteDb

async def my_workflow(session_state, input_data: str):
    # Step 1: Process input
    result1 = await agent1.arun(input_data)

    # Step 2: Process result
    result2 = await agent2.arun(result1.content)

    return result2.content

workflow = Workflow(
    name="My Workflow",
    steps=my_workflow,
    db=SqliteDb(db_file="tmp/workflow.db"),
)
```

Workflow Parameters

- **name** (str, required): Name of the workflow
- **steps** (callable, required): Function defining workflow steps
- **db** (BaseDb, optional): Database for persistence
- **description** (str, optional): Description of the workflow
- **session_state** (dict, optional): Initial session state

Workflow Methods

run()

Execute the workflow synchronously.

```
result = workflow.run(input_data="test")
```

arun()

Execute the workflow asynchronously.

```
result = await workflow.arun(input_data="test")
```

Workflow Example from Cookbook

```
"""🎓 Advanced Research Workflow - Your AI Research Assistant!

This advanced example demonstrates how to build a sophisticated blog post generator usi
the new workflow v2.0 architecture. The workflow combines web research capabilities wit
professional writing expertise using a multi-stage approach:

1. Intelligent web research and source gathering
2. Content extraction and processing
3. Professional blog post writing with proper citations

Key capabilities:
- Advanced web research and source evaluation
- Content scraping and processing
- Professional writing with SEO optimization
- Automatic content caching for efficiency
- Source attribution and fact verification

Run `pip install openai googlesearch-python newspaper4k lxml_html_clean sqlalchemy agno
"""

import asyncio
import json
from textwrap import dedent
from typing import Dict, Optional

from agno.agent import Agent
from agno.db.sqlite import SqliteDb
from agno.models.openai import OpenAIChat
from agno.tools.googlesearch import GoogleSearchTools
from agno.tools.newspaper4k import Newspaper4kTools
from agno.utils.log import logger
from agno.utils.pprint import pprint_run_response
from agno.workflow.workflow import Workflow
from pydantic import BaseModel, Field


# --- Response Models ---
class NewsArticle(BaseModel):
    title: str = Field(..., description="Title of the article.")
    url: str = Field(..., description="Link to the article.")
```

```
    summary: Optional[str] = Field(
        ..., description="Summary of the article if available."
    )


class SearchResults(BaseModel):
    articles: list[NewsArticle]


class ScrapedArticle(BaseModel):
    title: str = Field(..., description="Title of the article.")
    url: str = Field(..., description="Link to the article.")
    summary: Optional[str] = Field(
        ..., description="Summary of the article if available."
    )
    content: Optional[str] = Field(
        ...,
        description="Full article content in markdown format. None if content is unavai
    )


# --- Agents ---
research_agent = Agent(
    name="Blog Research Agent",
    model=OpenAIChat(id="gpt-4o-mini"),
    tools=[GoogleSearchTools()],
    description=dedent("""\
    You are BlogResearch-X, an elite research assistant specializing in discovering
    high-quality sources for compelling blog content. Your expertise includes:

    - Finding authoritative and trending sources
    - Evaluating content credibility and relevance
    - Identifying diverse perspectives and expert opinions
    - Discovering unique angles and insights
    - Ensuring comprehensive topic coverage
    """),
    instructions=dedent("""\
    1. Search Strategy 🔍
       - Find 10-15 relevant sources and select the 5-7 best ones
       - Prioritize recent, authoritative content
       - Look for unique angles and expert insights
```

```
    2. Source Evaluation 📊
       - Verify source credibility and expertise
       - Check publication dates for timeliness
       - Assess content depth and uniqueness
    3. Diversity of Perspectives 🌐
       - Include different viewpoints
       - Gather both mainstream and expert opinions
       - Find supporting data and statistics
    """),
    output_schema=SearchResults,
)

content_scraper_agent = Agent(
    name="Content Scraper Agent",
    model=OpenAIChat(id="gpt-4o-mini"),
    tools=[Newspaper4kTools()],
    description=dedent("""\
    You are ContentBot-X, a specialist in extracting and processing digital content
    for blog creation. Your expertise includes:

    - Efficient content extraction
    - Smart formatting and structuring
    - Key information identification
    - Quote and statistic preservation
    - Maintaining source attribution
    """),
    instructions=dedent("""\
    1. Content Extraction 📑
       - Extract content from the article
       - Preserve important quotes and statistics
       - Maintain proper attribution
       - Handle paywalls gracefully
    2. Content Processing 🔄
       - Format text in clean markdown
       - Preserve key information
       - Structure content logically
    3. Quality Control ✅
       - Verify content relevance
       - Ensure accurate extraction
       - Maintain readability
    """),
```

```
    output_schema=ScrapedArticle,
)

blog_writer_agent = Agent(
    name="Blog Writer Agent",
    model=OpenAIChat(id="gpt-4o"),
    description=dedent("""\
    You are BlogMaster-X, an elite content creator combining journalistic excellence
    with digital marketing expertise. Your strengths include:

    - Crafting viral-worthy headlines
    - Writing engaging introductions
    - Structuring content for digital consumption
    - Incorporating research seamlessly
    - Optimizing for SEO while maintaining quality
    - Creating shareable conclusions
    """),
    instructions=dedent("""\
    1. Content Strategy 📝
       - Craft attention-grabbing headlines
       - Write compelling introductions
       - Structure content for engagement
       - Include relevant subheadings
    2. Writing Excellence ✍️
       - Balance expertise with accessibility
       - Use clear, engaging language
       - Include relevant examples
       - Incorporate statistics naturally
    3. Source Integration 🔍
       - Cite sources properly
       - Include expert quotes
       - Maintain factual accuracy
    4. Digital Optimization 💻
       - Structure for scanability
       - Include shareable takeaways
       - Optimize for SEO
       - Add engaging subheadings

    Format your blog post with this structure:
    # {Viral-Worthy Headline}
```

```
    ## Introduction
    {Engaging hook and context}

    ## {Compelling Section 1}
    {Key insights and analysis}
    {Expert quotes and statistics}

    ## {Engaging Section 2}
    {Deeper exploration}
    {Real-world examples}

    ## {Practical Section 3}
    {Actionable insights}
    {Expert recommendations}

    ## Key Takeaways
    - {Shareable insight 1}
    - {Practical takeaway 2}
    - {Notable finding 3}

    ## Sources
    {Properly attributed sources with links}
    """),
    markdown=True,
)


# --- Helper Functions ---
def get_cached_blog_post(session_state, topic: str) -> Optional[str]:
    """Get cached blog post from workflow session state"""
    logger.info("Checking if cached blog post exists")
    return session_state.get("blog_posts", {}).get(topic)


def cache_blog_post(session_state, topic: str, blog_post: str):
    """Cache blog post in workflow session state"""
    logger.info(f"Saving blog post for topic: {topic}")
    if "blog_posts" not in session_state:
        session_state["blog_posts"] = {}
    session_state["blog_posts"][topic] = blog_post
```

```
def get_cached_search_results(session_state, topic: str) -> Optional[SearchResults]:
    """Get cached search results from workflow session state"""
    logger.info("Checking if cached search results exist")
    search_results = session_state.get("search_results", {}).get(topic)
    if search_results and isinstance(search_results, dict):
        try:
            return SearchResults.model_validate(search_results)
        except Exception as e:
            logger.warning(f"Could not validate cached search results: {e}")
    return search_results if isinstance(search_results, SearchResults) else None


def cache_search_results(session_state, topic: str, search_results: SearchResults):
    """Cache search results in workflow session state"""
    logger.info(f"Saving search results for topic: {topic}")
    if "search_results" not in session_state:
        session_state["search_results"] = {}
    session_state["search_results"][topic] = search_results.model_dump()


def get_cached_scraped_articles(
    session_state, topic: str
) -> Optional[Dict[str, ScrapedArticle]]:
    """Get cached scraped articles from workflow session state"""
    logger.info("Checking if cached scraped articles exist")
    scraped_articles = session_state.get("scraped_articles", {}).get(topic)
    if scraped_articles and isinstance(scraped_articles, dict):
        try:
            return {
                url: ScrapedArticle.model_validate(article)
                for url, article in scraped_articles.items()
            }
        except Exception as e:
            logger.warning(f"Could not validate cached scraped articles: {e}")
    return scraped_articles if isinstance(scraped_articles, dict) else None


def cache_scraped_articles(
    session_state, topic: str, scraped_articles: Dict[str, ScrapedArticle]
):
```

```
    """Cache scraped articles in workflow session state"""
    logger.info(f"Saving scraped articles for topic: {topic}")
    if "scraped_articles" not in session_state:
        session_state["scraped_articles"] = {}
    session_state["scraped_articles"][topic] = {
        url: article.model_dump() for url, article in scraped_articles.items()
    }


async def get_search_results(
    session_state, topic: str, use_cache: bool = True, num_attempts: int = 3
) -> Optional[SearchResults]:
    """Get search results with caching support"""

    # Check cache first
    if use_cache:
        cached_results = get_cached_search_results(session_state, topic)
        if cached_results:
            logger.info(f"Found {len(cached_results.articles)} articles in cache.")
            return cached_results

    # Search for new results
    for attempt in range(num_attempts):
        try:
            print(
                f"🔍 Searching for articles about: {topic} (attempt {attempt + 1}/{num_
            )
            response = await research_agent.arun(topic)

            if (
                response
                and response.content
                and isinstance(response.content, SearchResults)
            ):
                article_count = len(response.content.articles)
                logger.info(f"Found {article_count} articles on attempt {attempt + 1}")
                print(f"✅ Found {article_count} relevant articles")

                # Cache the results
                cache_search_results(session_state, topic, response.content)
                return response.content
```

```
            else:
                logger.warning(
                    f"Attempt {attempt + 1}/{num_attempts} failed: Invalid response typ
                )

        except Exception as e:
            logger.warning(f"Attempt {attempt + 1}/{num_attempts} failed: {str(e)}")

    logger.error(f"Failed to get search results after {num_attempts} attempts")
    return None


async def scrape_articles(
    session_state,
    topic: str,
    search_results: SearchResults,
    use_cache: bool = True,
) -> Dict[str, ScrapedArticle]:
    """Scrape articles with caching support"""

    # Check cache first
    if use_cache:
        cached_articles = get_cached_scraped_articles(session_state, topic)
        if cached_articles:
            logger.info(f"Found {len(cached_articles)} scraped articles in cache.")
            return cached_articles

    scraped_articles: Dict[str, ScrapedArticle] = {}

    print(f"📄 Scraping {len(search_results.articles)} articles...")

    for i, article in enumerate(search_results.articles, 1):
        try:
            print(
                f"📖 Scraping article {i}/{len(search_results.articles)}: {article.titl
            )
            response = await content_scraper_agent.arun(article.url)

            if (
                response
                and response.content
```

```
                and isinstance(response.content, ScrapedArticle)
            ):
                scraped_articles[response.content.url] = response.content
                logger.info(f"Scraped article: {response.content.url}")
                print(f"✅ Successfully scraped: {response.content.title[:50]}...")
            else:
                print(f"❌ Failed to scrape: {article.title[:50]}...")

        except Exception as e:
            logger.warning(f"Failed to scrape {article.url}: {str(e)}")
            print(f"❌ Error scraping: {article.title[:50]}...")

    # Cache the scraped articles
    cache_scraped_articles(session_state, topic, scraped_articles)
    return scraped_articles


# --- Main Execution Function ---
async def blog_generation_execution(
    session_state,
    topic: str = None,
    use_search_cache: bool = True,
    use_scrape_cache: bool = True,
    use_blog_cache: bool = True,
) -> str:
    """
    Blog post generation workflow execution function.

    Args:
        session_state: The shared session state
        topic: Blog post topic (if not provided, uses execution_input.input)
        use_search_cache: Whether to use cached search results
        use_scrape_cache: Whether to use cached scraped articles
        use_blog_cache: Whether to use cached blog posts
    """

    blog_topic = topic

    if not blog_topic:
        return "❌ No blog topic provided. Please specify a topic."
```

```
    print(f"🎨 Generating blog post about: {blog_topic}")
    print("=" * 60)

    # Check for cached blog post first
    if use_blog_cache:
        cached_blog = get_cached_blog_post(session_state, blog_topic)
        if cached_blog:
            print("📋 Found cached blog post!")
            return cached_blog

    # Phase 1: Research and gather sources
    print("\n🔍 PHASE 1: RESEARCH & SOURCE GATHERING")
    print("=" * 50)

    search_results = await get_search_results(
        session_state, blog_topic, use_search_cache
    )

    if not search_results or len(search_results.articles) == 0:
        return f"❌ Sorry, could not find any articles on the topic: {blog_topic}"

    print(f"📊 Found {len(search_results.articles)} relevant sources:")
    for i, article in enumerate(search_results.articles, 1):
        print(f"   {i}. {article.title[:60]}...")

    # Phase 2: Content extraction
    print("\n📄 PHASE 2: CONTENT EXTRACTION")
    print("=" * 50)

    scraped_articles = await scrape_articles(
        session_state, blog_topic, search_results, use_scrape_cache
    )

    if not scraped_articles:
        return f"❌ Could not extract content from any articles for topic: {blog_topic}

    print(f"📖 Successfully extracted content from {len(scraped_articles)} articles")

    # Phase 3: Blog post writing
    print("\n✍️ PHASE 3: BLOG POST CREATION")
    print("=" * 50)
```

```
    # Prepare input for the writer
    writer_input = {
        "topic": blog_topic,
        "articles": [article.model_dump() for article in scraped_articles.values()],
    }

    print("🤖 AI is crafting your blog post...")
    writer_response = await blog_writer_agent.arun(json.dumps(writer_input, indent=2))

    if not writer_response or not writer_response.content:
        return f"❌ Failed to generate blog post for topic: {blog_topic}"

    blog_post = writer_response.content

    # Cache the blog post
    cache_blog_post(session_state, blog_topic, blog_post)

    print("✅ Blog post generated successfully!")
    print(f"📝 Length: {len(blog_post)} characters")
    print(f"📚 Sources: {len(scraped_articles)} articles")

    return blog_post


# --- Workflow Definition ---
blog_generator_workflow = Workflow(
    name="Blog Post Generator",
    description="Advanced blog post generator with research and content creation capabi
    db=SqliteDb(
        session_table="workflow_session",
        db_file="tmp/blog_generator.db",
    ),
    steps=blog_generation_execution,
    session_state={},  # Initialize empty session state for caching
)


if __name__ == "__main__":
    import random
```

```
    async def main():
        # Fun example topics to showcase the generator's versatility
        example_topics = [
            "The Rise of Artificial General Intelligence: Latest Breakthroughs",
            "How Quantum Computing is Revolutionizing Cybersecurity",
            "Sustainable Living in 2024: Practical Tips for Reducing Carbon Footprint",
            "The Future of Work: AI and Human Collaboration",
            "Space Tourism: From Science Fiction to Reality",
            "Mindfulness and Mental Health in the Digital Age",
            "The Evolution of Electric Vehicles: Current State and Future Trends",
            "Why Cats Secretly Run the Internet",
            "The Science Behind Why Pizza Tastes Better at 2 AM",
            "How Rubber Ducks Revolutionized Software Development",
        ]

        # Test with a random topic
        topic = random.choice(example_topics)

        print("🧪 Testing Blog Post Generator v2.0")
        print("=" * 60)
        print(f"📝 Topic: {topic}")
        print()

        # Generate the blog post
        resp = await blog_generator_workflow.arun(
            topic=topic,
            use_search_cache=True,
            use_scrape_cache=True,
            use_blog_cache=True,
        )

        pprint_run_response(resp, markdown=True, show_time=True)

    asyncio.run(main())
```

6. Knowledge & VectorDB APIs

6.1 Knowledge Class

The `Knowledge` class provides agentic RAG capabilities.

Import

```
from agno.knowledge.knowledge import Knowledge
```

Basic Knowledge Setup

```
from agno.knowledge.knowledge import Knowledge
from agno.vectordb.lancedb import LanceDb, SearchType
from agno.knowledge.embedder.openai import OpenAIEmbedder

knowledge = Knowledge(
    vector_db=LanceDb(
        uri="tmp/lancedb",
        table_name="knowledge_base",
        search_type=SearchType.hybrid,
        embedder=OpenAIEmbedder(id="text-embedding-3-small"),
    ),
)
```

Knowledge Methods

add_documents()

Add documents to the knowledge base.

```
knowledge.add_documents([
    "Document 1 content",
    "Document 2 content",
])
```

add_urls()

Add documents from URLs.

```
knowledge.add_urls([
    "https://example.com/doc1",
    "https://example.com/doc2",
])
```

search()

Search the knowledge base.

```
results = knowledge.search("query", num_documents=5)
```

7. Database APIs

7.1 Database Overview

Agno supports multiple database backends for persistence.

Supported Databases

- **PostgreSQL** (Production recommended)
- **SQLite** (Development only)
- **MongoDB**
- **MySQL**
- **Redis**
- **DynamoDB**
- **Firestore**
- **SingleStore**
- **SurrealDB**
- **JSON files**
- **In-memory**

PostgreSQL (Production)

```
from agno.db.postgres import PostgresDb

db = PostgresDb(
    db_url="postgresql+psycopg://user:password@localhost:5432/dbname"
)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=db,
)
```

SQLite (Development)

```
from agno.db.sqlite import SqliteDb

db = SqliteDb(db_file="tmp/agents.db")

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=db,
)
```

8. Model APIs

8.1 Model Overview

Agno is model-agnostic and supports 30+ model providers.

Supported Providers

- OpenAI
- Anthropic (Claude)
- Google (Gemini)
- AWS Bedrock

- Azure OpenAI
- Groq
- Together AI
- Fireworks
- Mistral
- Cohere
- Perplexity
- xAI
- And more...

OpenAI

```
from agno.models.openai import OpenAIChat

model = OpenAIChat(id="gpt-4o")
agent = Agent(model=model)
```

Anthropic Claude

```
from agno.models.anthropic import Claude

model = Claude(id="claude-sonnet-4-5")
agent = Agent(model=model)
```

9. Tool APIs

9.1 Tool Overview

Agno provides 100+ built-in toolkits with thousands of tools.

Built-in Toolkits

- **Web Search**: DuckDuckGo, Google Search, Exa
- **Finance**: YFinance, Financial Modeling Prep

- **Code**: GitHub, GitLab, Code Execution
- **Data**: SQL, CSV, Excel, JSON
- **APIs**: REST, GraphQL
- **Media**: Image generation, Video processing
- **And many more...**

Using Toolkits

```
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.tools.yfinance import YFinanceTools

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[
        DuckDuckGoTools(),
        YFinanceTools(stock_price=True, company_info=True),
    ],
)
```

Custom Tools

Create custom tools using the `Function` class:

```
from agno.tools.function import Function
from pydantic import BaseModel, Field

class WeatherInput(BaseModel):
    city: str = Field(..., description="City name")

def get_weather(city: str) -> str:
    '''Get weather for a city'''
    return f"Weather in {city}: Sunny, 72°F"

weather_tool = Function(
    name="get_weather",
    description="Get weather information for a city",
    fn=get_weather,
    input_schema=WeatherInput,
)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[weather_tool],
)
```

10. AgentOS API

10.1 AgentOS Overview

AgentOS is a FastAPI-based runtime for serving agents in production.

Basic Setup

```
from agno.os import AgentOS
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.db.postgres import PostgresDb

# Create agents
agent = Agent(
    name="My Agent",
    model=OpenAIChat(id="gpt-4o"),
    db=PostgresDb(db_url="..."),
)

# Create AgentOS
agent_os = AgentOS(agents=[agent])
app = agent_os.get_app()

# Run
if __name__ == "__main__":
    agent_os.serve(app="my_app:app", reload=True)
```

AgentOS Parameters

- **agents** (list[Agent], optional): List of agents
- **teams** (list[Team], optional): List of teams
- **workflows** (list[Workflow], optional): List of workflows
- **db** (BaseDb, optional): Shared database
- **description** (str, optional): Description of the OS

API Endpoints

Run Agent

```
POST /agents/{agent_id}/runs
```

Run Team

```
POST /teams/{team_id}/runs
```

Run Workflow

```
POST /workflows/{workflow_id}/runs
```

11. Memory & Session Management

11.1 Memory Management

Memory allows agents to store and recall user-specific information.

Basic Memory Setup

```
from agno.memory import MemoryManager

memory = MemoryManager()

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    memory=memory,
    add_memories_to_context=True,
)
```

11.2 Session Management

Sessions track conversations and maintain state.

Using Sessions

```
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=SqliteDb(db_file="tmp/agents.db"),
    user_id="user-123",
    session_id="session-456",
    add_history_to_context=True,
    num_history_runs=5,
)

# Session state is automatically managed
response = agent.run("Hello!")
```

12. Guardrails & Security

12.1 Guardrails Overview

Guardrails provide validation and security for agents.

Built-in Guardrails

- **PII Detection**: Detect personally identifiable information
- **Prompt Injection**: Detect prompt injection attacks
- **OpenAI Moderation**: Use OpenAI's moderation API

Using Guardrails

```
from agno.guardrails.pii import PIIDetection
from agno.guardrails.prompt_injection import PromptInjectionDetection

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    guardrails=[
        PIIDetection(),
        PromptInjectionDetection(),
    ],
)
```

13. Cookbook Examples

The Agno cookbook contains hundreds of examples demonstrating various features and use cases. All examples are available in the `/cookbook` directory.

13.1 Getting Started Examples

The getting started cookbook provides step-by-step examples:

1. **Basic Agent** - Creating your first agent
2. **Agent with Tools** - Adding web search and other tools
3. **Agent with Knowledge** - Setting up RAG with knowledge bases
4. **Write Your Own Tool** - Creating custom tools
5. **Structured Output** - Using Pydantic models for type-safe outputs
6. **Agent with Storage** - Adding database persistence
7. **Agent State** - Managing session state
8. **Agent Context** - Dynamic context management
9. **Agent Session** - Session management
10. **User Memories** - Storing and recalling user information
11. **Retry Function Calls** - Error handling and retries
12. **Human-in-the-Loop** - User confirmation workflows
13. **Image Agent** - Multimodal image processing

14. **Generate Image** - Image generation
15. **Generate Video** - Video generation
16. **Audio Input/Output** - Audio processing
17. **Agent Team** - Multi-agent collaboration
18. **Research Agent** - Advanced research workflows
19. **Blog Generator Workflow** - Complete workflow example

13.2 Advanced Agent Examples

Agentic Search

- Agentic RAG with hybrid search
- RAG with reasoning capabilities
- Infinity reranker integration

Async Operations

- Basic async agent execution
- Concurrent tool calls
- Parallel agent execution
- Async streaming

Caching

- Model response caching
- Async caching
- Stream caching

Context Management

- Dynamic instructions
- Few-shot learning
- Location-aware instructions
- DateTime context

Custom Logging

- Custom logging handlers

- File logging
- Advanced logging configurations

Events Handling

- Basic agent events
- Reasoning agent events
- Custom event handlers

Multimodal Agents

- Image processing
- Video generation
- Audio processing
- File handling

RAG Implementation

- Basic RAG setup
- Advanced RAG patterns
- Knowledge base management

Session Management

- Session state
- Session history
- Session summaries

State Management

- Agent state
- Workflow state
- Team state

13.3 Database Examples

Examples for all supported databases: - PostgreSQL - SQLite - MongoDB - MySQL - Redis - DynamoDB - Firestore - SingleStore - SurrealDB - JSON files - In-memory storage

13.4 Model Examples

Examples for all supported model providers: - OpenAI - Anthropic (Claude) - Google (Gemini) - AWS Bedrock - Azure OpenAI - Groq - Together AI - Fireworks - Mistral - Cohere - Perplexity - xAI - And many more...

13.5 Tool Examples

Examples for built-in toolkits: - Web Search (DuckDuckGo, Google, Exa) - Finance (YFinance, Financial Modeling Prep) - Code (GitHub, GitLab) - Data (SQL, CSV, Excel) - APIs (REST, GraphQL) - Media (Image, Video, Audio) - MCP Tools

13.6 Team Examples

- Basic team setup
- Multi-agent coordination
- Team with shared state
- Nested teams

13.7 Workflow Examples

- Sequential workflows
- Parallel execution
- Conditional steps
- Loops and branches
- Complex multi-step workflows

13.8 Knowledge Examples

- Setting up knowledge bases
- Adding documents
- URL ingestion
- PDF processing
- Hybrid search
- Reranking

- All vector database integrations

13.9 Memory Examples

- User memory management
- Memory search
- Memory updates
- Memory deletion
- Memory in context

13.10 Guardrails Examples

- PII detection
- Prompt injection detection
- OpenAI moderation
- Custom guardrails

13.11 Integration Examples

- Discord integration
- Slack integration
- Custom integrations

13.12 Evaluation Examples

- Accuracy evaluation
- Performance benchmarking
- Reliability testing
- Custom evaluation metrics

14. Best Practices

14.1 Agent Creation

CRITICAL: Never create agents in loops

```
# ❌ WRONG - Recreates agent every time (significant overhead)
for query in queries:
    agent = Agent(...)  # DON'T DO THIS

# ✅ CORRECT - Create once, reuse
agent = Agent(...)
for query in queries:
    agent.run(query)
```

14.2 Database Selection

- **Development**: Use SQLiteDb for simplicity
- **Production**: Always use PostgresDb for scalability and reliability

14.3 Knowledge Base Setup

Always set `search_knowledge=True` when using Knowledge:

```
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    knowledge=knowledge,
    search_knowledge=True,  # Critical: enables agentic RAG
    instructions="Use knowledge base, cite sources"
)
```

14.4 Structured Output

Use `output_schema` for predictable, composable behavior:

```
from pydantic import BaseModel

class Result(BaseModel):
    summary: str
    findings: list[str]

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    output_schema=Result,
)
result: Result = agent.run(query).content
```

14.5 Error Handling

Always wrap agent.run() in try-except blocks:

```
try:
    response = agent.run(query)
except Exception as e:
    logger.error(f"Agent error: {e}")
    # Handle error appropriately
```

14.6 Performance Optimization

- Reuse agents across requests
- Use async methods (arun) for concurrent operations
- Cache knowledge base results when appropriate
- Use session caching for frequently accessed sessions